

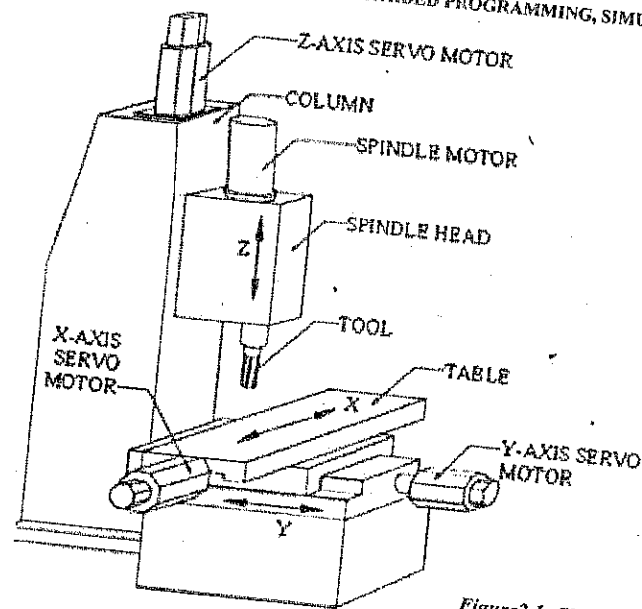


Figure 2.1: Sketch of a Numerical Control Technology (Source: CAD/CAM/CIM)

the necessary inputs are sent to the X and Y-axis servomotors. These motors drive the respective slides to the commanded position. When the distance information from the feedback devices equals the programmed values the slide movement stops. The input is then given to the Z-axis servomotor to perform the drilling operation. The system switches on the motor driving the spindle carrying the drill and commands the Z-axis servomotor to move at the programmed feedrate into the workpiece till the programmed depth is reached. The Z axis feedback transducer ensures the correct depth and feed rate. The drill is then withdrawn to the desired height and the machine starts the positioning movements to the next location of the drill. This procedure thus ensures production of accurate workpieces.

2.1 Types of CNC Machines

Although numerical control has caused very little change in basic machine types and operations, it has placed heavier demands on the machines since the cutting tool is brought to its machining position much more efficient than if it were done manually by a machine operator. Because of this the machines are now actually spending more time per shift cutting than in the past. This higher percentage of cutting time results in faster wear on the ways, gears and screws – which leads to shorter machine life as measured by years.

Early machine tools were designed and arranged for ease of operation by the man standing directly in front. With numerical control this is no longer a valid design criteria since the operator no longer controls cutter movement.

While the average machine tool may be removing material only 20% of the time, with the addition of an electronic control this may be easily increased

In order to carry out the milling operation on the workpiece the coordinate information (X, and Y coordinates) of the starting point and the ending point has to be coded in the NC program. Similarly, the information regarding the direction of rotation and speed of the spindle, use of coolant and the feed rate is also coded suitably. The CNC controller decodes the positioning information coded in the NC program and the slide is moved to the programmed position at the required feed rate. Each slide is fitted with a feedback transducer, which continuously monitors the slide position and compares with the programmed position as well as the feed rate. The feedback transducer is mounted either on the slide or on the servomotor and measures the displacement or position of the slide.

For example, let us assume that a hole is to be drilled at the location $X = 100.0$ and $Y = 150.0$. The corresponding block of the program is read by the control system and

to between 70% and 80%. Obviously, such a controller adds to the cost of the machine; in fact, sometimes the controller may even be more expensive than the machine itself.

Historically, machine tools were kept as simple as possible in order to keep their cost down. A high quality, low labor rate factor was required for individual machining operations. Currently, since the cost of labor has risen at a rate which is considerably higher than the increase in machine tool costs, there is pressure to add many features to the machine tools that yield maximum output to each man-hour of operation.

An excellent example of the influence NC has had on machine tool development is the slant bed of the turret lathe which enables removed metal to fall through the machine rather than to build up around the workpiece as it would on a conventional lathe.

One of the simplest and most prevalent of all numerically controlled machine tools is the single-spindle drilling machine. While it is true that many drills are program controlled on three axes – the XY table movement and the Z motion of the spindle advance and retract – it still remains that a large percentage of numerically controlled machine tools is represented by the two-axis XY control. Machine tool designers have recently been combining functions which were in the past done on separate machines. The best example of this may be seen in the machining center which is capable of milling, drilling, spotting, threading, tapping and boring on up to five surfaces in one setup.

Computer numerical control is applied to a variety of machines. Most of these find ready application in aircraft, automobile and general engineering industry. Some of them are listed below:

- i. Machining Centre
Horizontal

Vertical
Universal

- ii. CNC Lathes
- iii. CNC Turning Centers
- iv. Turn-mill Centers
- v. CNC Milling/Drilling Machines, Plane Milling Machines
- vi. Gear Hobbing Machines
- vii. Gear Shaping Machines
- viii. Wire Cut EDM/EDM
- ix. Tube Bending
- x. Electron Beam Welding
- xi. Laser/Arc/Plasma Cutting
- xii. Co-ordinate Measuring Machines
- xiii. Grinding Machines
Surface Grinder
Cylindrical Grinder
Centre less Grinder
- xiv. Tool and Cutter Grinder
- xv. CNC Boring and Jig Boring Machines
- xvi. PCB Drilling Machines
- xvii. Press Brakes
- xviii. CNC Guillotines
- xix. CNC Transfer Lines, SPM's
- xx. Electrochemical Milling Machines
- xxi. Abrasive Water Jet Cutting Machines
- xxii. Flow Forming Machines
- xxiii. Roll Forming Machines
- xxiv. Turret Punch Press

2.2 CNC Programming

Because of the added feature of a computer's memory, CNC programming is a lot easier and simpler than conventional NC programming. Most CNC machines can be programmed using a standardized list of codes that were developed by the Electronics Industries Association (EIA). Some of these functions include cutting an arc in a clockwise direction (G02-circular interpolation) and telling the machine that the dimensions will be given in millimeters (G71-metric programming).

Table 1: Electronic Industries Association (EIA) Codes

Code	Description	
Preparatory Functions		
G00	Denotes rapid traverse for point-to-point positioning.	5-49
G01	Linear interpolation	50-59
G02	Circular interpolation clockwise	60-69
G03	Circular interpolation counterclockwise	70
G04	Dwell	71
G05-07	Unassigned	72
G08	Acceleration at a smooth rate	73
G09	Deceleration at a smooth rate	74
G10-16	Unassigned	75
G13-16	Axis selection codes	76-79
G17 XY	plane selection	80
G18 ZX	plane selection	81
G19 YZ	plane selection	82
G20-32	Unassigned	83
G33	Thread cutting, constant lead	84
G34	Thread cutting, increasing lead	85-89
		90

Thread cutting, decreasing lead
Unassigned
Cutter diameter compensation cancels
Cutter diameter compensation left
Cutter diameter compensation right
Cutter diameter compensation inside corner (used to adjust for differences in programmed and actual cutter size)
Cutter diameter compensation outside corner (used to adjust for differences in programmed and actual cutter size)
Unassigned
Used with adaptive controls
Unassigned
Inch programming
Metric programming
Three-dimensional circular interpolation clockwise
Three-dimensional circular interpolation counterclockwise
Multi-quadrant circular interpolations cancel
Multi-quadrant circular interpolation
Unassigned
Cycle cancel
Drill cycle
Drill cycle with dwell
Intermittent or deep hole drilling cycle
Tapping cycle
Boring cycles
Absolute positioning

G91	Incremental positioning
G92	Register preload code
G93	Inverse time feed rate
G94	Inches (millimeters) per minute feed rate
G95	Inches (millimeters) per revolution feed rate
G96	Unassigned
G97	Revolutions per minute spindle speed
G98-99	Unassigned

Miscellaneous Functions

Code	Description
M00	Program stop
M01	Optional (planned) stop
M02	End of program
M03	Spindle on clockwise
M04	Spindle counterclockwise
M05	Spindle off
M06	Tool change
M07	Coolant on (flood)
M08	Coolant on (mist)
M09	Coolant off
M10	Automatic clamp
M11	Automatic unclamp
M12	Synchronize multiple axes
M13	Spindle clockwise and coolant on
M14	Spindle counterclockwise and coolant on
M15	Rapid motion positive direction
M16	Rapid motion negative direction

M17-18	Unassigned
M19	Spindle orient and stop
M20-29	Unassigned
M30	End of tape (will rewind tape automatically)
M31	Interlock bypass
M32-39	Unassigned
M40-46	Gear changes if used; otherwise unassigned
M47	Continues program execution from start of program
M48	Cancel M47
M49	Deactivate manual speed or feed override
M50-57	Unassigned
M58	Cancel M59
M59	RPM hold
M60-99	Unassigned

Other Addresses

A	Rotary motion about the x-axis
B	Rotary motion about the y-axis
C	Rotary motion about the z-axis
D	Angular dimension around a special axis (also used for a third feed function)
E	Angular dimension around a special axis or special feed function
H	Unassigned
I	X-axis arc center point
J	Y-axis arc center point
K	Z-axis arc center point
L	Unassigned

- O Used on some controllers in place of N address of sequence numbers
- P Special rapid traverse code or third axis parallel to the x-axis
- Q Special rapid traverse code or third axis parallel to the y-axis
- R Special rapid traverse code or third axis parallel to the z-axis (also used for radius designation)
- U Secondary axis parallel to x-axis
- V Secondary axis parallel to y-axis
- W Secondary axis parallel to z-axis

2.3 CAD/CAM

The advances in computer technology have dramatically revolutionized the designing and manufacturing of a part. A computer aided design/computer-aided manufacturing (CAD/CAM) system allows a single person to draw a part three dimensionally, select tools for machining it, create tool paths by picking points on the edge of a part, and download it to a machining center for immediate processing.

The post processor for a CAD/CAM system can automatically write the CNC program for the machine tool in a matter of seconds.

A new product can literally be taken from design board to the shop floor in a matter of minutes. Because CAD/CAM software can automatically create the program with a post processor, a skilled machinist with computer skills is no longer needed. It should be noted, however, that sometimes the software adds additional moves and cuts that can be omitted by an experienced CNC programmer. Removing unnecessary lines of code could save machining time and money if large quantities of parts are being

produced. Therefore, there is still a need for at least one experienced machinist with CNC programming skills for each company that produces large numbers of parts using CNC equipment.

Review Questions

1. Distinguish between servo motor and stepper motor used to drive axis of NC machines.
2. Explain why ball screws are recycled in NC nut assembly.
3. Differentiate between G and M codes used for NC programming.

Step 9: If the graph is not correct according to the program, correct the program and run it again.

Example 8.2.2.: Linear Interpolation 2

The component shown in figure 8.2.2 below is to be machined out of 20mm diameter fibre material, using C-502 CNC lathe machine. Use the G&M-code language to write a complete turning program according to the drawing. Use 2000rev/min as spindle speed, 150mm/min as feed and 2mm as depth of cut at a time.

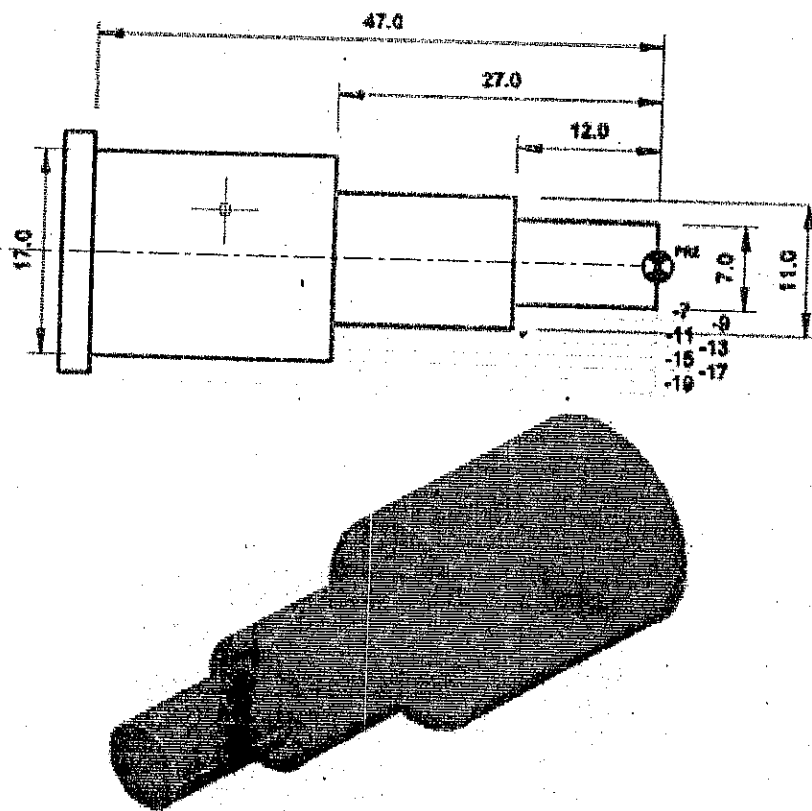


Figure 8.2.2: Turning using linear interpolation (2)

Solution

For latching software and initial setup refer to example 8.2.1 (section 8.2).

8.2.4 Generated program (2)

The program is generated based on the information contained in Figure 8.2.2 of example 8.2.2.

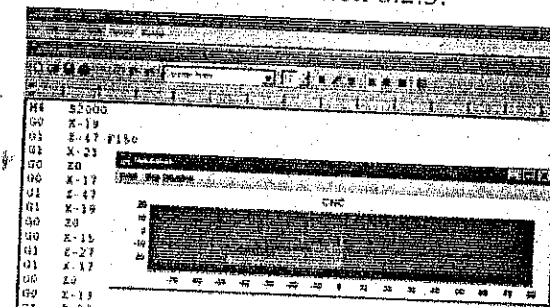
Program

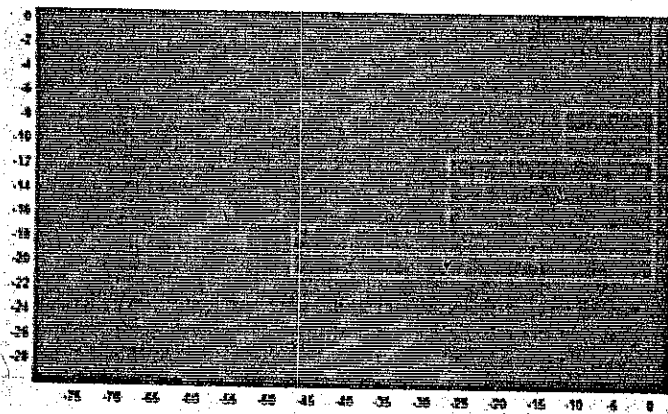
M4	S2000		
G0	X-19	G1	X-15
G1	Z-47 F150	G0	Z0
G1	X-21	G0	X-11
G0	Z0	G1	Z-27
G0	X-17	G1	X-13
G1	Z-47	G0	Z0
G1	X-19	G0	X-9
G0	Z0	G1	Z-12
G0	X-15	G1	X-11
G1	Z-27	G0	Z0
G1	X-17	G0	X-7
G0	Z0	G1	Z-12
G0	X-13	G1	X-9
G1	Z-27	G0	Z0
		M5	

8.2.5 Procedure (2)

Follow procedures as explained in section 8.2.3.

Simulation



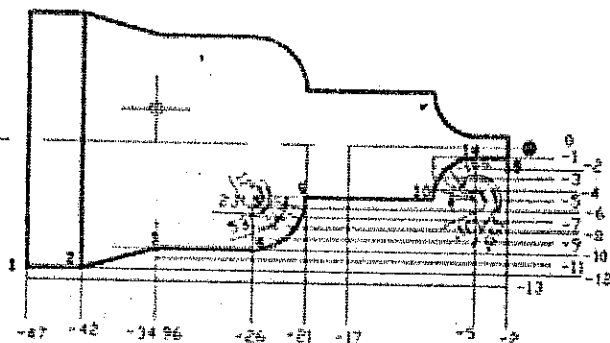


Simulation 8.2.1: Simulations (Linear interpolation)

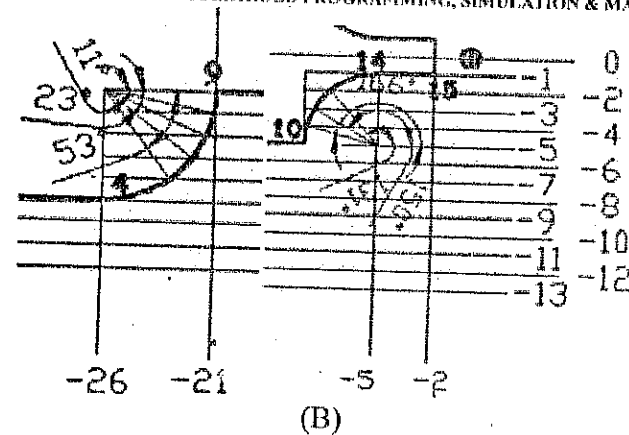
8.3 Examples on multiple turning with circular interpolation

Example 8.3.1: Multiple turning with circular interpolation 1

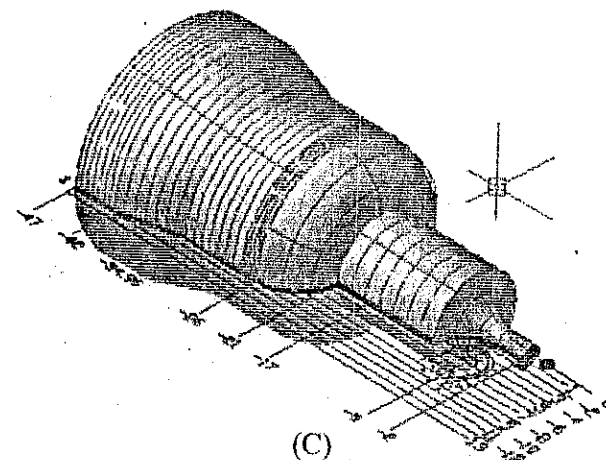
The component shown in figure 8.3.1 below is to be machined out of 20mm diameter aluminum material, using C-502 CNC lathe machine. Use the G&M-code language to write a complete turning program according to the drawing. Use 3000rev/min as spindle speed, 200mm/min as feed and 1mm as depth of cut at a time.



(A)



(B)



(C)

Figure 8.3.1: Multiple turning with circular interpolation (I)

Solution

For latching software and initial setup refer to example 8.2.1 (section 8.2).

8.3.1 Generated program.(I)

The program is generated based on the information contained in Figure 8.3.1 of example 8.3.1.

Refer to section 7.3 (5 & 6) which fully defines:

G02 – Clockwise circular interpolation

G03 – Counter clockwise circular interpolation, and their components.

Program	Description
M4 S2500	Start spindle at 2500 rpm
G0 X-12	Rapid move for 1 st cut
G1 Z-47	Cut through point 2&1
G0 X-13	Retract 1mm
G0 Z0	Rapid move to X-13, Z0
G0 X-11	Rapid move for 2 nd cut
G1 Z-38.39	Cut to a point b/w 2&3
G0 X-12	Retract 1mm
G0 Z0	Rapid move to X-12, Z0
G0 X-10	Rapid move for 3 rd cut
G1 Z-34.96	Cut to a point 3
G1 X-12 Z-42	Cut from point 3 to 2
G0 X-13	Retract 1mm
G0 Z0	Rapid move to X-13, Z0
G0 X-9	Rapid move for 4 th cut
G1 Z-21	Cut to Z-21
G02 I-5 K-26 R5 C-53 E-90	Re locates from PRZ to LRZ and cut arc from -53° to -90°.
G0 X-10	Retract 1mm
G0 Z0	Rapid move to X-10, Z0
G0 X-8	Rapid move for 5 th cut
G1 Z-21	Cut to Z-21
G02 I-5 K-26 R5 C-37 E-53	Re locates from PRZ to LRZ and cut arc from -37° to -53°
G0 X-9	Retract 1mm
G0 Z0	Rapid move to X-9, Z0
G0 X-7	Rapid move for 6 th cut
G1 Z-21	Cut to Z-21

G02 I-5 K-26 R5 C-23 E-37	Re locates from PRZ to LRZ and cut arc from -23° to -37°
G0 X-8	Retract 1mm
G0 Z0	Rapid move to X-8, Z0
G0 X-6	Rapid move for 7 th cut
G1 Z-21	Cut to Z-21
G02 I-5 K-26 R5 C-11 E-23	Re locates from PRZ to LRZ and cut arc from -11° to -23°
G0 X-7	Retract 1mm
G0 Z0	Rapid move to X-7, Z0
G0 X-5	Rapid move for 8 th cut
G1 Z-21	Cut to Z-21
G02 I-5 K-26 R5 C0 E-11	Re locates from PRZ to LRZ and cut arc from 0° to -11°
G0 X-6	Retract 1mm
G0 Z0	Rapid move to X-6, Z0
G0 X-4	Rapid move for 9 th cut
G1 Z-5	Cut to Z-5
G03 I-5 K-5 R4 C166 E180	Re locates from PRZ to LRZ and cut arc from 166° to 180°
G0 X-5	Retract 1mm
G0 Z0	Rapid move to X-5, Z0
G0 X-3	Rapid move for 10 th cut
G1 Z-5	Cut to Z-5
G03 I-5 K-5 R4 C150 E188	Re locates from PRZ to LRZ and cut arc from 150° to 188°
G0 X-4	Retract 1mm
G0 Z0	Rapid move to X-4, Z0

G0 X-2 Rapid move for 11th cut
 G1 Z-5 Cut to Z-5
 G03 I-5 K-5 R4 C131 E150
 Re locates from PRZ to LRZ and
 cut arc from 131° to 150°
 G0 X-3 Retract 1mm
 G0 Z0 Rapid move to X-3, Z0
 G0 X-1 Rapid move for 12th cut
 G1 Z-5 Cut to Z-5
 G03 I-5 K-5 R4 C90 E131

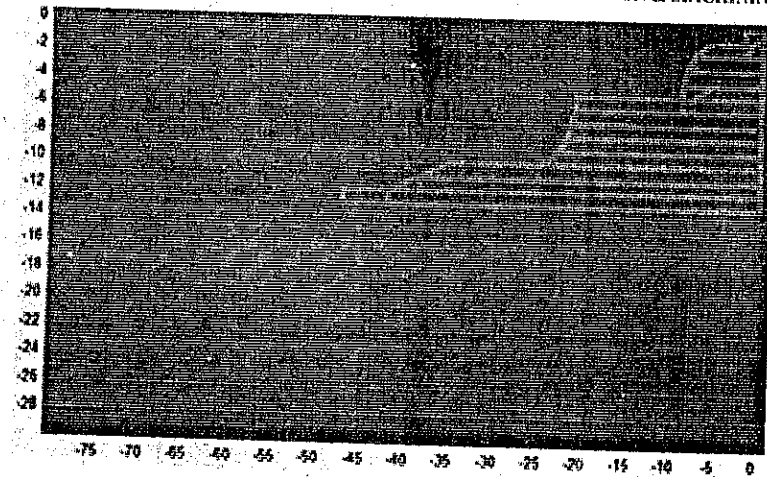
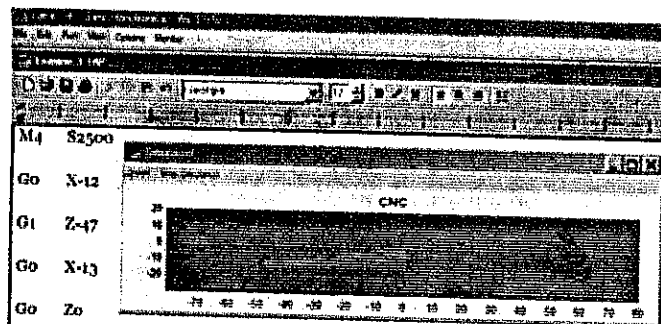
Re locates from PRZ to LRZ and
 cut arc from 90° to 131°

G0 X-2 Retract 1mm
 G0 Z0 Rapid move to X-2, Z0
 G0 X0 Rapid moves to Z0, X0
 G1 Z-2 Cut to Z-2
 G1 X-2 Cut to X-2
 G0 X0 Z0 Rapid move to X0, Z0
 M5 Stop spindle

8.3.2 Procedure (1)

Follow procedures as explained in section 8.2.3.

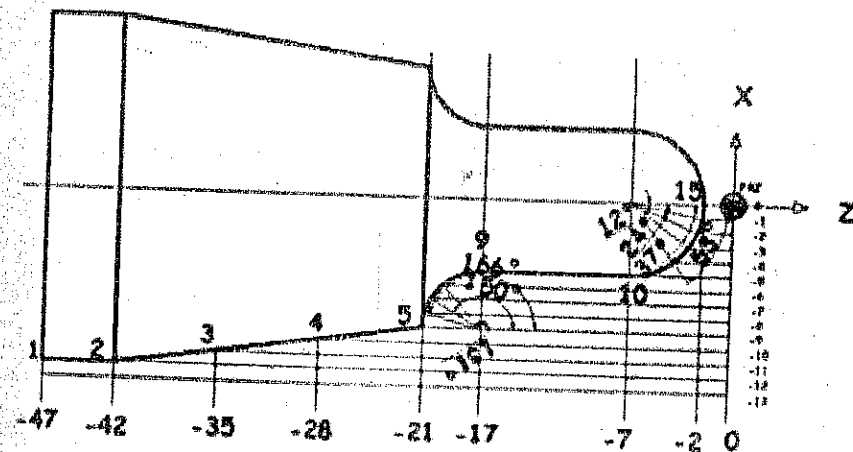
Simulations



Simulation 8.3.1.: Simulations on multiple turning with circular interpolation (1)

Example 8.3.2.: Multiple turning with circular interpolation 2

The component shown in figure 8.3.3 below is to be machined out of 20mm diameter aluminum material, using C-502 CNC lathe machine. Use the G&M-code language to write a complete turning program according to the drawing. Use 3000rev/min as spindle speed, 200mm/min as feed and 1mm as depth of cut at a time.



A **MACRO** is more or less an abbreviation for some lines of code that are used more than once or twice. A FORM is a local **subroutine** (which can be called external). A FUNCTION is (more or less) a **subroutine** that is called external. If the **subroutine** is called external (used by more than one program) use a FUNCTION

✓ A routine is an Object Script program. It can include one or more procedures, subroutines, and functions, as well as any combination of the three.
Routines

A routine is a callable block of user-written code that is an Object Script program. A routine performs commonly needed operations. Its name is determined by the name of the .MAC file that you choose for saving it. Depending on if a routine returns a value, you can invoke a routine with one or both of the following sets of syntax:

```
DO ^RoutineName
SET x = $$RoutineName
```

A routine is defined within a namespace. You can use an extended routine reference to execute a user-defined routine defined in a namespace other than the current namespace:

```
DO "SAMPLES"RoutineName
```

Generally, routines serve as containers for subroutines, methods, and procedures.

The routine is identified by a label (also referred to as a tag) at the beginning of the block of code. This label is the name of the routine. This label is (usually) followed by parentheses which contain a list of parameters to be passed from the calling program to the routine.

When you save a routine to a file, the file name cannot include the underscore (" _"), hyphen ("-"), or semicolon (" ;") characters; names that include such characters are not valid.

Subroutines

A subroutine is a named block of code within a routine. Typically, a subroutine begins with label and ends with a **QUIT** statement. It can accept parameters and does not return a value. To invoke a subroutine, use the following syntax:

```
DO SubroutineRoutine
```

where *Subroutine* is a code block within the *Routine* file (**Routine.MAC**).

The form of a subroutine is:

The form of a subroutine is:

```
Label(parameters) // comment  
// code  
QUIT // note that QUIT has no arguments
```

For more details on subroutines, see the section below on subroutines as legacy code.

If you enclose the code and **QUIT** statement within curly braces, the subroutine is a procedure and can be treated as such.

Functions

Caché comes with many system-supplied functions (sometimes known as “intrinsic” functions), which are described in the ObjectScript Language Reference. This section describes user-defined (“extrinsic”) functions.

✓ A function is a named block of code within a routine. Typically, a function begins with label and ends with a **QUIT** statement. It can accept parameters and can also return a value. To invoke a function, there are two valid forms of the syntax:

```
SET rval= $$Function() /* returning a value */  
DO Function^Routine /* ignoring the return value */
```

where *Function* is a code block within the *Routine* file (**Routine.MAC**). In both syntactic forms you can use an extended routine reference to execute a function located in a different namespace.

The form of a function is:

```
Label(parameters)  
// code  
QUIT Return Value
```

If you enclose the code and **QUIT** statement within curly braces, the function is a procedure and can be treated as such. Note that because a procedure is private by default, you may wish to specify the **PUBLIC** keyword, as follows:

```
Label(parameters) PUBLIC {  
// code  
QUIT Return Value }
```

The following example defines a simple function (MyFunc) and calls it, passing two parameters and receiving a return value:

```

Main;
TRY{
  KILL x
  SET x=$$MyFunc(7,10)
  WRITE"returned value is "x,!
  RETURN
}
CATCH{WRITE$ZERROR,!
}
MyFunc(a,b)
SET c=a+b
QUIT c

```

The code invoking the function can ignore the return value, but a function's **QUIT** command must specify a return value. Attempting to exit a function with an argumentless **QUIT** generates a **<COMMAND>** error. The **<COMMAND>** error specifies the location of the call that invoked the function, followed by a message that specifies the offset location of the argumentless **QUIT** command within the called function. Refer to **\$ZERROR** for further details.

subroutine

~~A routine or subroutine, also referred to as a function, procedure, and subprogram, is a portion of code that may be called and executed anywhere in a program. For example, a routine may be used to save a file or display the time. ... Below is a basic example of a Perl subroutine.~~

Subroutine

A routine or subroutine, also referred to as a **function, procedure, and subprogram**, is a portion of code that may be called and executed anywhere in a program. For example, a routine may be used to save a file or display the time. Instead of writing the code each time these commonly-performed tasks are needed, routines are created and called when these tasks need to be performed. Below is a basic example of a Perl subroutine.

Subroutine example

```
&hello;  
  
sub hello {  
  
    print "Hello World!\n";  
  
}
```

In the above example, a user could call the **hello** subroutine (in this example, by typing **&hello;**) anywhere in the program and have the program print **Hello World!** After the subroutine has completed, the program goes to the return address, which is directly after **&hello;** and runs any additional code.

Robot software is the set of coded commands or instructions that tell a mechanical device and electronic system, known together as a robot, what tasks to perform. Robot software is used to perform autonomous tasks. Many software systems and frameworks have been proposed to make programming robots easier.

Some robot software aims at developing intelligent mechanical devices. Common tasks include feedback loops, control, pathfinding, data filtering, locating and sharing data.

Industrial robot software

Software for industrial robots consists of data objects and lists of instructions, known as program flow (list of instructions). For example,

Go to Jig1

is an instruction to the robot to go to positional data named Jig1. Of course, programs can also contain implicit data for example

Tell axis 1 move 30 degrees.

Data and program usually reside in separate sections of the robot controller memory. One can change the data without changing the program and vice versa. For example, one can write a different program using the same Jig1 or one can adjust the position of Jig1 without changing the programs that use it.

Examples of programming languages for industrial robots

Due to the highly proprietary nature of robot software, most manufacturers of robot hardware also provide their own software. While this is not unusual in other automated control systems, the lack of standardization of programming methods for robots does pose certain challenges. For example, there are over 30 different manufacturers of industrial robots, so there are also 30 different robot programming languages required. There are enough similarities between the different robots that it is possible to gain a broad-based understanding of robot programming without having to learn each manufacturer's proprietary language.^[1]

One method of controlling robots from multiple manufacturers is to use a Post processor and Off-line programming (robotics) software. With this method, it is possible to handle brand-specific robot programming language from a universal programming language, such as Python (programming language).^[2] However, compiling and uploading fixed off-line code to a robot controller doesn't allow the robotic system to be state aware, so it cannot adapt its motion and recover as the environment changes. Unified real-time adaptive control for any robot is currently possible with a few different third-party tools.

Some examples of published robot programming languages are shown below.

Task in plain English:

Move to P1 (a general safe position)
 Move to P2 (an approach to P3)
 Move to P3 (a position to pick the object)
 Close gripper
 Move to P4 (an approach to P5)
 Move to P5 (a position to place the object)
 Open gripper
 Move to P1 and finish

A scripting language is a high-level programming language that is used to control the software application, and is interpreted in real-time, or "translated on the fly", instead of being compiled in advance. A scripting language may be a general-purpose programming language or it may be limited to specific functions used to augment the running of an application or system program. Some scripting languages, such as RoboLogix, have data objects residing in registers, and the program flow represents the list of instructions, or instruction set, that is used to program the robot.

Programming languages in industrial robotics^[4]

Robot brand	Language name
ABB	<u>RAPID</u>
Comau	PDL2
Fanuc	<u>Karel</u>
Kawasaki	AS

Kuka	KRL
<u>Stäubli</u>	VAL3
Yaskawa	Inform

Programming languages are generally designed for building data structures and algorithms from scratch, while scripting languages are intended more for connecting, or "gluing", components and instructions together. Consequently, the scripting language instruction set is usually a streamlined list of program commands that are used to simplify the programming process and provide rapid application development.

Robot application software

Regardless which language is used; the end result of robot software is to create robotic applications that help or entertain people. Applications include command-and-control and tasking software. Command-and-control software includes robot control GUIs for tele-operated robots, point-n-click command software for autonomous robots, and scheduling software for mobile robots in factories. Tasking software includes simple drag-n-drop interfaces for setting up delivery routes, security patrols and visitor tours; it also includes custom programs written to deploy specific applications. General purpose robot application software is deployed on widely distributed robotic platforms.

Safety considerations

Programming errors represent a serious safety consideration, particularly in large industrial robots. The power and size of industrial robots mean they are capable of inflicting severe injury if programmed incorrectly or used in an unsafe manner. Due to the mass and high-speeds of industrial robots, it is always unsafe for a human to remain in the work area of the robot during automatic operation. The system can begin motion at unexpected times and a human will be unable to react quickly enough in many situations, even if prepared to do so. Thus, even if the software is free of programming errors, great care must be taken to make an industrial robot safe for human workers or human interaction, such as loading or unloading parts, clearing a part jam, or performing maintenance. The *ANSI/RIA*

R15.06-1999 American National Standard for Industrial Robots and Robot Systems - Safety Requirements (revision of ANSI/ R15.06-1992) book from the Robotic Industries Association is the accepted standard on robot safety. This includes guidelines for both the design of industrial robots, and the implementation or integration and use of industrial robots on the factory floor. Numerous safety concepts such as safety controllers, maximum speed during a teach mode, and use of physical barriers are covered.

Programming languages

The most popular language in **robotics** is probably C/C++ (C++ is an object-oriented successor to the C language). Python is also very popular due to its **use** in machine learning and also because it **can be used** to develop ROS packages

Programming skills for robotics

Programming is a key skill to develop for working in robotics. In this step, we'll introduce programming, the important languages, and how you can get started with tools like Arduino and Raspberry Pi.

What is a computer program?

Robots process sensor data, perform cognition and plan actions using computer programs that are executed on a processor. Computer programs are essentially a set of instructions that operate on an input to produce an output.

Example: A face recognition program in a robot will:

1. take an image of a person as an input,
2. scan the image for a specific set of features,
3. compare these features to a library of known faces,
4. find a match, then
5. return the name of the person as an output.

The program will perform exactly these same set of instructions every time it executes.

Most programming languages are written in normal text, which is easy for humans to understand. Programs are then compiled into machine code for a processor to execute (or byte code, for a virtual machine to execute).

Programming languages

There are numerous programming languages available, e.g. C/C++, Java, Fortran, Python etc. The most popular language in robotics is probably C/C++ (C++ is an object-oriented successor to the C language). Python is also very popular due to its use in machine learning and also because it can be used to develop ROS packages - see below.

There are additional, important software tools used in robotics, in particular:

- **Robot Operating System (ROS)** is a set of software libraries and tools that helps you build robot applications. You can also write your own programs for ROS e.g. in C/C++ or Python.
- **Matlab** which is used for data analysis and interfaces with ROS (also, Octave is a free, open-source equivalent to Matlab [available here.](#))

C programming and the Arduino microcontroller

The C/C++ language is one of the most widely used programming languages in robotics. The Arduino microcontroller uses a programming language based on C and is a great way to learn the basics of this important language whilst doing hands-on robotics.

Robotics is a sub-domain of engineering and science that includes mechanical engineering, electrical engineering, computer science, and others. Robotics deals with the design, construction, operation, and use of robots and computer systems for their control, sensory feedback, and information processing. A robot is a unit that implements this interaction with the physical world based on sensors, actuators, and information processing. Industry is a key application of robots, or to be precise Industry 4.0, where industrial robots are used.

Service robots

A **service robot** is a machine that provides services for humans. A distinction is made between use for private individuals and professional applications. Vacuum

cleaner and lawn mower robots have become established in the private sphere. Here are the different types of service robots:

Domestic robots

Security robots

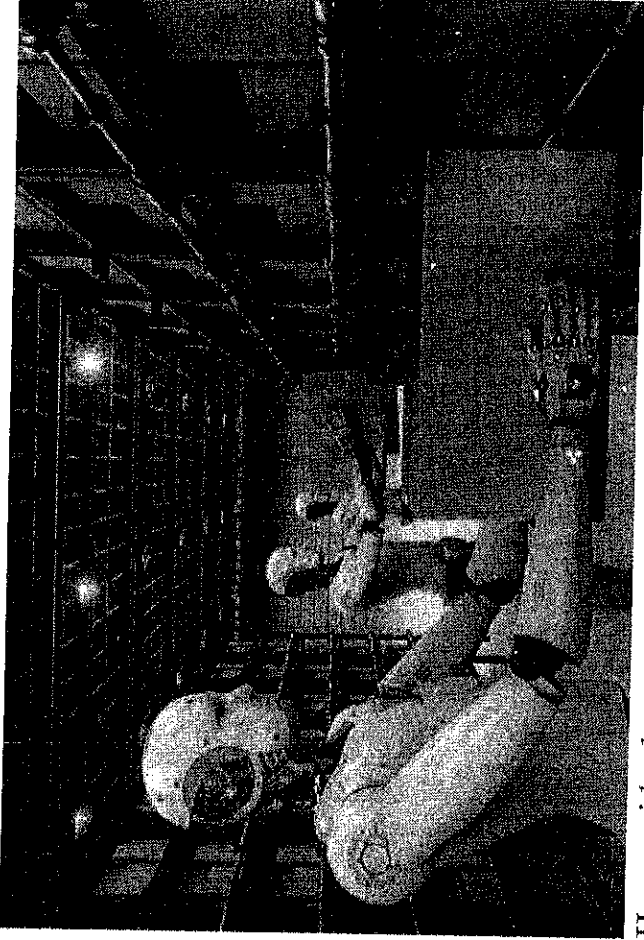
Robots in the hospitality industry

Robots in the agriculture industry

Robots in medicine

Robots in surgery

Toy robots



Humanoid robots

Humanoid robots are machines that are designed to look like humans. Joint positions and movements are inspired by the human locomotor system. This is also clear by the fact that humanoid robots usually move on two legs in an upright position. The main motive for research and development in the field of humanoid robots is artificial intelligence (AI).

21/12/21/2002

Zunmy's copy

Yashor
17/11/23

28

INDUSTRIAL ROBOTS

An industrial robot is comprised of a robot manipulator, power supply, and controllers. Robotic manipulators can be divided into two sections, each with a different function: Robot Arm and Body. The arm and body of a robot are used to move and position parts or tools within a work envelope. They are formed from three joints connected by large links. Most industrial robots work in auto assembly lines, putting cars together. Robots can do a lot of this work more efficiently than human beings because they are so precise.

Robot is are programmable, multifunctional manipulator designed to move material, part, tools or specialized devices through variable programmed motions for the performances of variety of tasks.

Types of robots

- (i) Manipulator robot.
- (ii) Legged/ wheeled robot.
- (iii) Autonomous underwater vehicle/ Unmanned Aerial vehicle.

Robot Characteristics

In this section an industrial robot is considered to be an open kinematic chain of rigid bodies called links, interconnected by joints with actuators to drive them. A robot can also be viewed as a set of integrated subsystems.

1. Manipulator. The mechanical structure that performs the actual work of the robot, consisting of links and joints with actuators.
2. Feedback devices. Transducers that sense the position of various linkages and/or joints that transmit this information to the controller.
3. Controller. Computer used to generate signals for the drive system so as to reduce response error in positioning and applying force during robot assignments.
4. Power source. Electric, pneumatic, and hydraulic power systems used to provide and regulate the energy needed for the manipulator's actuators.

Types of robots by application

Nowadays, robots do a lot of different tasks in many fields and the number of jobs entrusted to robots is growing steadily. That's why in my opinion one of the best ways how to divide robots into types is a division by their application.

There are:

***Industrial robots** - Industrial robots are robots used in an industrial manufacturing environment. Usually these are articulated arms specifically developed for such applications as welding, material handling, painting and others. If we judge purely by application this type could also include some automated guided vehicles and other robots.

***Domestic or household robots** - Robots used at home. This type of robots includes many quite different devices such as robotic vacuum cleaners, robotic pool cleaners, sweepers, gutter cleaners and other robots that can do different chores. Also, some surveillance and telepresence robots could be regarded as household robots if used in that environment.

***Medical robots** - Robots used in medicine and medical institutions. First and foremost - surgery robots. Also, some automated guided vehicles and maybe lifting aides.

***Service robots** - Robots that don't fall into other types by usage. These could be different data gathering robots, robots made to show off technologies, robots used for research, etc.

***Military robots** - Robots used in military. This type of robots includes bomb disposal robots, different transportation robots, reconnaissance drones. Often robots initially created for military purposes can be used in law enforcement, search and rescue and other related fields.

***Entertainment robots** - These are robots used for entertainment. This is a very broad category. It starts with toy robots such as RoboBabies or the running alarm clock and ends with real heavyweights such as articulated robot arms used as motion simulators.

***Space robots** - I'd like to single out robots used in space as a separate type. This type would include robots used on the International Space Station, Canadarm that was used in Shuttles, as well as Mars rovers and other robots used in space.

***Hobby and competition robots** - Robots that you create. Line followers, sumo-bots, robots made just for fun and robots made for competition.

Manipulator elements

It consists of two or three compact joints. Robot manipulators are created from a sequence of link and joint combinations. The axes are the movable components of the robotic manipulator that cause relative motion between adjoining links. An industrial robot is comprised of a robot manipulator, power supply and controllers.

The mechanical joints used to construct the robotic arm manipulator consist of five principal types. Two of the joints are linear, in which the relative motion between adjacent links is non-rotational, and three are rotary types, in which the relative motion involves rotation between links.

The arm-and-body section of robotic manipulators is based on one of four configurations. Each of these anatomies provides a different work envelope and is suited for different applications.

Gantry Robots

These robots have linear joints and are mounted overhead. They are also called Cartesian and rectilinear robots.

Cylindrical Robots

Named for the shape of its work envelope, cylindrical anatomy robots are fashioned from linear joints that connect to a rotary base joint.

Polar Robots

The base joint of a polar robot allows for twisting and the joints are a combination of rotary and linear types. The work space created by this configuration is spherical.

Jointed-Arm Robots

This is the most popular industrial robotic configuration. The arm connects with a twisting joint, and the links within it are connected with rotary joints. It is also called an articulated robot.

Types of robot sensors

There are different sensors to choose from and we will identify the characteristics of few sensors, and also understand why and where they are used.

Light sensor

A Light sensor is used to detect light and create a voltage difference. The two main light sensors generally used in robots are Photoresistor and Photovoltaic cells. Other kinds of light sensors like Phototubes, Phototransistors, CCD's etc. are rarely used.

Photoresistor is a type of resistor whose resistance varies with change in light intensity; more light leads to less resistance and less light leads to more resistance. These inexpensive sensors can be easily implemented in most light dependent robots.

Photovoltaic cells convert solar radiation into electrical energy. This is especially helpful if you are planning to build a solar robot. Although photovoltaic cell is considered as an energy source, an intelligent implementation combined with transistors and capacitors can convert this into a sensor.

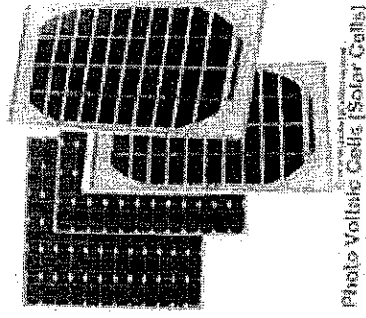
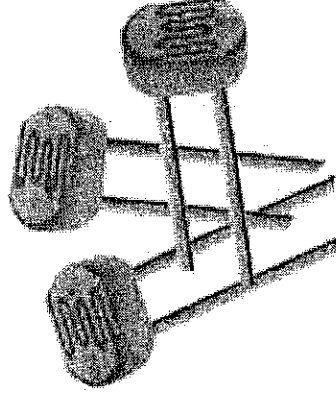


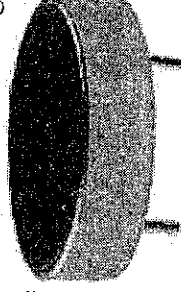
Photo-Voltaic Cells (Solar Cells)



Photoresistors (LDR)

Sound sensor

As the name suggests, this sensor (generally a microphone) detects sound and returns a voltage proportional to the sound level. A simple robot can be designed to navigate based on the sound it receives. Imagine a robot which turns right for one clap and turns left for two claps. Complex robots can use the same microphone for speech and voice recognition.

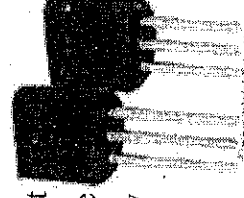


Sound Sensor (Mic)

Implementing sound sensors is not as easy as light sensors because Sound sensors generate a very small voltage difference which should be amplified to generate measurable voltage change.

Temperature sensor

What if your robot has to work in a desert and transmit ambient temperature? Simple solution is to use a temperature sensor. Tiny temperature sensor ICs provide voltage difference for a change in temperature. Few generally used temperature sensor IC's are LM34, LM35, TMP35, TMP36, and TMP37.



Temperature Sensors

Contact sensor

Contact sensors are those which require physical contact against other objects to trigger. A push button switch, limit switch or tactile bumper switch are all examples of contact sensors. These sensors are mostly used for obstacle avoidance robots. When these switches hit an obstacle, it triggers the robot to do a task, which can be reversing, turning, switching on a LED, Stopping etc. There are also capacitive contact sensors which react only to human touch (Not sure if they react to animals touch).



Limit Switch (Contact Sensor)

Touch screen Smart phones available these days use capacitive touch sensors (Not to be confused with older stylus based models). Contact Sensors can be easily implemented, but the drawback is that they require physical contact. In other words, your robot will not turn until it hits an object. A better alternative is to use a proximity sensor.

Proximity sensor

This is a type of sensor which can detect the presence of a nearby object within a given distance, without any physical contact. The working principle of a Proximity sensor is simple. A transmitter transmits an electromagnetic radiation or creates an electrostatic field and a receiver receives and analyzes the return signal for interruptions. There are different types of Proximity sensors and we will discuss only a few of them which are generally used in robots.

1. **Infrared (IR) Transceivers:** An IR LED transmits a beam of IR light and if it finds an obstacle, the light is simply reflected back which is captured by an IR receiver. Few IR transceivers can also be used for distance measurement.
2. **Ultrasonic Sensor:** These sensors generate high frequency sound waves; the received echo suggests an object interruption. Ultrasonic Sensors can also be used for distance measurement.
3. **Photoresistor:** Photoresistor is a light sensor; but, it can still be used as a proximity sensor. When an object comes in close proximity to the sensor, the amount of light changes which in turn changes the resistance of the Photoresistor. This change can be detected and processed.

There are many different kinds of proximity sensors and only a few of them are generally preferred for robots. For example, Capacitive Proximity sensors are available which detects change in capacitance around it. Inductive proximity sensor detects objects and distance through the use of induced magnetic field.

Distance sensor

Most proximity sensors can also be used as distance sensors, or commonly known as **Range Sensors**; IR transceivers and Ultrasonic Sensors are best suited for distance measurement

1. **Ultrasonic Distance Sensors:** The sensor emits an ultrasonic pulse and is captured by a receiver. Since the speed of sound is almost constant in air, which is 344m/s , the time between send and receive is calculated to give the distance between your robot and the obstacle. Ultrasonic distance sensors are especially useful for underwater robots.

2. **Infrared Distance sensor:** IR circuits are designed on triangulation principle for distance measurement. A transmitter sends a pulse of IR signals which is detected by the receiver if there is an obstacle and based on the angle the signal is received, distance is calculated. SHARP has a family of IR transceivers which are very useful for distance measurement. A simple transmit and receive using a couple of transmitters and receivers will still do the job of distance measurement, but if you require precision, then prefer the triangulation method

3. **Laser range Sensor:** Laser light is transmitted and the reflected light is captured and analyzed. Distance is measured by calculating the speed of light and time taken for the light to reflect back to the receiver. These sensors are very useful for longer distances.

4. **Encoders:** These sensors (not actually sensors, but a combination of different components) convert angular position of a shaft or wheel into an analog or digital code. The most popular encoder is an optical encoder which includes a rotational disk, light source and a light detector (generally an IR transmitter and IR receiver). The rotational disk has transparent and opaque pattern (or just black and white pattern) painted or printed over it. When the disk rotates along with the wheel the emitted light is interrupted generating a signal output. The number of times the interruption happens and the diameter of the wheel can together give the distance travelled by the robot.

5. **Stereo Camera:** Two cameras placed adjacent to each other can provide depth information using its stereo vision. Processing the data received from a camera is difficult for a robot with minimal processing power and memory. If opted for, they make a valuable addition to your robot.

There are other stretch and bend sensors which are also capable of measuring distance. But, their range is so limited that they are almost useless for mobile robots.

pressure sensor

As the name suggests, pressure sensor measures pressure. Tactile pressure sensors are useful in robotics as they are sensitive to touch, force and pressure. If you design a robot hand and need to measure the amount of grip and pressure required to hold an object, then this is what you would want to use.

Tilt sensors

Tilt sensors measure tilt of an object. In a typical analog tilt sensor, a small amount of mercury is suspended in a glass bulb. When mercury flows towards one end, it closes a switch which suggests a tilt.

Navigation / Positioning sensors

The name says it all. Positioning sensors are used to approximate the position of a robot, some for indoor positioning and few others for outdoor positioning.

- **GPS (Global Positioning System):** The most commonly used positioning sensor is a GPS. Satellites orbiting our earth transmit signals and a receiver on a robot acquires these signals and processes it. The processed information can be used to determine the approximate position and velocity of a robot. These GPS systems are extremely helpful for outdoor robots, but fail indoors. They are also bit expensive at the moment and if their prices fall, very soon you would see most robots with a GPS module attached.
- **Digital Magnetic Compass:** Similar to a handheld magnetic compass, Digital Magnetic compass provides directional measurements using the earth's magnetic field which guides your robot in the right direction to reach its destination. These sensors are cheap compared to GPS modules, but a compass works best along with a GPS module if you require both positional feedback and navigation. Philips KMZ51 is sensitive enough to detect earth's magnetic field.
- **Localization:** Localization refers to the task of automatically determining the location of a robot in complex environment. Localization is based on external elements called landmarks which can be either artificially placed landmarks, or natural landmark. In the first approach, artificial landmarks or beacons are placed around the robot, and a robot's sensor captures these signals to determine its exact location. Natural landmarks can be doors, windows, walls, etc. which are sensed by a robots sensor / vision system (Camera). Localization can be achieved

using beacons which generate Wi-Fi, Bluetooth, Ultrasound, Infrared, Radio transmissions, Visible Light, or any similar signal.

Acceleration sensor

An accelerometer is a device which measures acceleration and tilt. There are two kinds of forces which can affect an accelerometer: Static force and Dynamic Force

- **Static Force:** Static force is the frictional force between any two objects. For example, earth's gravitational force is static which pulls an object towards it. Measuring this gravitational force can tell you how much your robot is tilting. This measurement is exceptionally useful in a balancing robot, or to tell you if your robot is driving uphill or on a flat surface.
- **Dynamic force:** Dynamic force is the amount of acceleration required to move an object. Measuring this dynamic force using an accelerometer tells you the velocity/speed at which your robot is moving. We can also measure vibration of a robot using an accelerometer, if in any case you need to.

Accelerometer comes in different flavors. Always select the one which is most appropriate for your robot. Some of the factors which you need to consider before selecting an accelerometer are:

1. Output Type: Analog or Digital
2. Number of Axis: 1, 2 or 3
3. Accelerometer Swing: $\pm 1.5g$, $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
4. Sensitivity: Higher or Lower (Higher the better)
5. Bandwidth

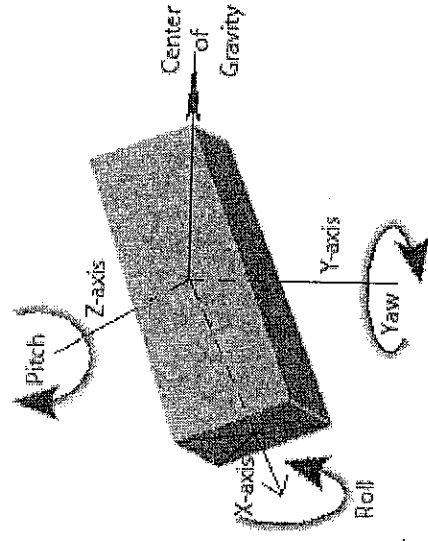
Gyroscope

A gyroscope or simply Gyro is a device which measures and helps maintain orientation using the principle of angular momentum. In other words, a Gyro is used to measure the rate of rotation around a particular axis. Gyroscope is especially useful when you want your robot to not depend on earth's gravity for maintaining Orientation. (Unlike accelerometer)

Inertial Measurement Units (IMU)

Inertial Measurement Units combine properties of two or more sensors such as Accelerometer, Gyro, Magnetometer, etc, to measure orientation, velocity and gravitational forces. In simple words, IMU's are capable of providing feedback by detecting changes in an objects orientation

(pitch, roll and yaw), velocity and gravitational forces. Few IMUs go a step further and combine a GPS device providing positional feedback.



Pitch, Roll and Yaw of an Object

Voltage sensors

Voltage sensors typically convert lower voltages to higher voltages, or vice versa. One example is a general Operational-Amplifier (Op-Amp) which accepts a low voltage, amplifies it, and generates a higher voltage output. Few voltage sensors are used to find the potential difference between two ends (Voltage Comparator). Even a simple LED can act as a voltage sensor which can detect a voltage difference and light up. (not considering current requirements here)

Current sensors

Current sensors are electronic circuits which monitor the current flow in a circuit and output either a proportional voltage or a current. Most current sensors output an analog voltage between 0V to 5V which can be processed further using a microcontroller.

Other sensors for robots

There are hundreds of sensors made today to sense virtually anything you can think of, and it is almost impossible to list all available sensors. Apart from those mentioned above, there are many other sensors used for specific applications. For example: Humidity Sensors measure Humidity; Gas sensors are designed to detect particular gases (helpful for robots which detect gas leaks); Potentiometers are so versatile that they can be used in numerous different applications; Magnetic Field Sensors detect the strength of magnetic field around it.

Sources of error in industrial robot. ✓

However, **industrial robots** do not often offer height accuracy. **Robotic** inaccuracy is attributed to several **error sources**, which are divided into three main categories: **Active joint errors**, **kinematic errors** and **non-kinematic errors**.

Survey of Sources of Error in Robot Machining Among the different performances related to the robot itself, precision is often used to describe its capabilities, and is further divided in: repeatability, accuracy and resolution. Repeatability and accuracy estimate the closeness between a set of attained positions and orientations of the TCP, when repeating the robot motions into the same

Environment dependent, robot dependent and process dependent errors are addressed. The second part analyses the errors according to their source, magnitude and frequency spectrum.

Many industrial applications require accurate robots to achieve critical tasks, such as aeronautic manufacturing and metrological inspection. However, industrial robots do not often offer height accuracy. Robotic inaccuracy is attributed to several error sources, which are divided into three main categories: Active joint errors, kinematic errors and non-kinematic errors. By active joints, we refer to the articulated and actuated joints of a robot, since some joints are articulated but not actuated, for example in parallel robots, these are called passive joints.

1- Joint errors

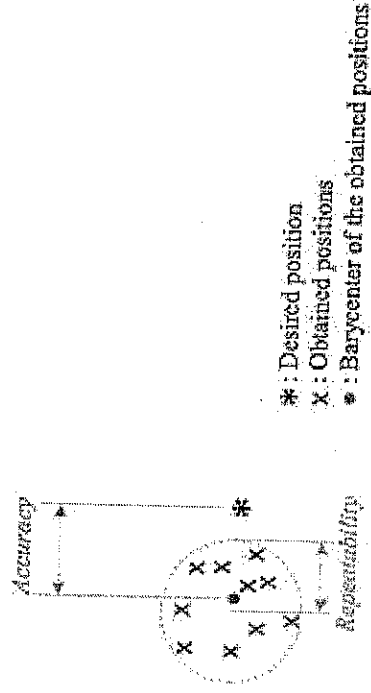
These errors represent the errors related to the displacement value provided by the encoders of the active joint(s) of the robot. They represent the differences between the movement reported by the sensor (i.e. an encoder) and the actual movement made by the joint. These differences are mainly caused by sensor errors themselves and the offset caused by the zeroing (or homing) of each active joint (i.e. the error in the zero or base position of the active joint).

2- Kinematic errors

The kinematic factors are related to the knowledge of the robot's kinematic model. A model that doesn't closely represent the actual geometry of the robot is a fundamental source of inaccuracy. The main causes of kinematic errors are summarized in the following:

- Differences between the nominal and actual lengths of the robot's links, which are mainly caused by manufacturing and assembly tolerances.
- Geometric characteristics of the robot's components (e.g. parallelism, orthogonality).
- Location errors of the reference frame(s): The robot base reference frame with respect to the work reference frame (also called cell frame or world frame) and the tool reference frame with respect to the flange frame (i.e. the last reference frame) of the robot.

- the flange frame (i.e. the last reference frame) of the robot.



3- Non-kinematic errors

The non-kinematic factors are attributed to the mechanical characteristics of the robot's components, which might be summarized in the following main elements:

- The stiffness of the mechanical components (e.g. the robot's links and gearboxes).
- The mechanical backlash (e.g. gearbox backlash).
- The effects of temperature on the robot's structure and mechanical components.

Knowing where to look and how to reduce your errors and improve your accuracy for any industrial application that requires accurate robots to achieve critical tasks will surely improve your production results. Use of a force torque sensor to add precision to your measurements might be one way to eliminate these pitfalls. Since careful measurement at each stage of the application process, while this might seem tedious during set up, will no doubt payoff in the long run.

Robotics programming

Robot software is the set of coded commands or instructions that tell a mechanical device and electronic system, known together as a robot, what tasks to perform. Robot software is used to perform autonomous tasks. Many software systems and frameworks have been proposed to make **programming robots** easier.

A robot program is a path in space to be followed by the manipulator, combined with peripheral actions that support the work cycle. To programme a robot, specific commands are entered into the robot's controller memory, and this action may be performed in a number of ways. For limited sequence robots programming occurs when limit switches and mechanical stops are set to control the endpoints of its motions. A sequencing device controls the occurrence of the motions, which in turn controls the movement of the joints that completes the motion cycle.

Two main programming languages are the best when used in robotics: **C++** and **Python**, often used together as each one has pros and cons. C++ is used in control loops, image processing and

to interface low-level hardware. **Python** is used to handle high-level behaviors and to quickly develop tests or proof of concepts.

Robot uses

- (i) Jobs that are dangerous for humans such as decontaminating robot which is use to cleaning the main circulating pump housing in the nuclear power plant.
- (ii) Repetitive jobs that are boring, stressful, or labor-intensive for humans such as welding robot.
- (iii) Menial tasks that human doesn't want to do like clearing of toilet whereby the scrub mate robot would do that.